CollegeHub

Deployment guide

From your laptop to GitHub to a public link, step by step, with every command written out.

Author	Aarif Mushtaq
Programme	B.Tech Artificial Intelligence and Data Science, University of Kashmir
Written	21 October 2026
Free-tier limits checked	21 October 2026 on render.com/docs/free and neon.com/pricing

What you will do

- **Part 1**: push the project to your own GitHub repository, in seven small stages with real commits.
- **Part 2**: run it for your demo on your laptop (the safest choice for a presentation).
- **Part 3**: publish it on the internet with Render (web app) and Neon (database), both free.
- **Part 4**: pre-demo checklist, security checklist and troubleshooting.

Choose your route. For the final presentation, run it **locally** (Part 2): no cold start, the QR camera works on *localhost*, and nothing can go wrong with free-tier limits. Use Part 3 when you need a public link to share. Part 3 has real limits (Render free spins down after 15 minutes, uploaded files vanish on restart, password-reset e-mail cannot be sent); they are listed honestly in the section itself.

Honest scope. I tested the Git commands in Part 1 in a scratch repository (all 139 files are pushed exactly once, the final tree is identical, the app starts after stage 4). I could not log in to your GitHub, Render or Neon accounts, so Parts 3 and 4 are written from the providers' current documentation and the project's own configuration; screens may look slightly different. The project's automated tests are run on the finished code, not on your hosted site.

Part 1. Push to GitHub (solo)

1.1 One-time setup

```
git --version            # install from https://git-scm.com if this fails
python --version         # 3.11 or newer

git config --global user.name  "Aarif Mushtaq"
git config --global user.email "<the e-mail on your GitHub account>"
git config --global init.defaultBranch main
git config --global pull.rebase false
git config --global core.autocrlf input      # Windows: use true
```

Windows: use **Git Bash** for every command in this guide. GitHub does not accept your password for *git push*; sign in with *gh auth login* (GitHub CLI), GitHub Desktop, or a personal access token (Settings → Developer settings → Personal access tokens, scope *repo*). Never commit or share a token.

```
gh auth login      # GitHub.com → HTTPS → follow the browser prompt
```

1.2 Unzip the project as a read-only source folder

```
cd ~
unzip /path/to/collegehub.zip -d ~/collegehub-unzip
mv ~/collegehub-unzip/collegehub ~/collegehub-source
ls ~/collegehub-source        # app  docs  tests  README.md  wsgi.py ...
```

You will copy files **out of** this folder into your repository folder, a few at a time, as you review them. You never run git in the source folder.

1.3 Create the empty GitHub repository

github.com → **New repository** → name **collegehub** → Private (or Public if your instructor wants to see it) → **do not** add a README, .gitignore or licence → **Create repository**.

1.4 First commit on main: the skeleton

```
cd ~
mkdir collegehub && cd collegehub
git init -b main
git remote add origin https://github.com/<YOUR-USERNAME>/collegehub.git

cd ~/collegehub-source
tar cf - .gitignore .env.example requirements.txt wsgi.py uploads/.gitkeep \
  | tar xf - -C ~/collegehub

cd ~/collegehub
git status
git add .gitignore .env.example requirements.txt wsgi.py uploads/.gitkeep
git commit -m "chore: add project skeleton"
git push -u origin main
```

The *.gitignore* is committed first so *.env*, databases and uploads can never be added by accident.

1.5 Protect main (optional but recommended)

Repository → **Settings** → **Branches** → **Add branch ruleset** for **main**: require a pull request before merging, block force pushes. Set required approvals to **0** (you are the only collaborator; GitHub will not let you approve your own pull request). After stage 6 has run once, you may also require the status check named **test**.

1.6 The seven stages

Each stage is one branch, a few commits, one pull request that you merge after reading **Files changed**. Do one or two stages per day and read the files you are pushing: every commit then reflects work you really did. Do not back-date commits or rewrite history.

Stage	Branch	Contains
1	feature/backend-core	Backend core and design system
2	feature/accounts-and-auth	Accounts, authentication, verification and public pages
3	feature/role-workspaces	Organizer and Faculty workspaces
4	feature/events-qr-ai	Events, QR attendance, student area, logistics, AI
5	feature/seed-data	Demo seed data
6	feature/tests-and-ci	Automated tests and CI
7	docs/documentation	Documentation and GitHub templates

The application first starts after **stage 4**; before that, some files import modules from later stages, so do not worry if *flask run* fails earlier.

Stage 1: Backend core and design system

Branch **feature/backend-core**

```
cd ~/collegehub
git checkout main
git pull
git checkout -b feature/backend-core

cd ~/collegehub-source
tar cf - \
    app/__init__.py \
    app/config.py \
    app/extensions.py \
    app/routes/__init__.py \
    app/services/__init__.py \
    app/constants.py \
    app/models.py \
    app/decorators.py \
    app/utils.py \
    app/cli.py \
    app/services/errors.py \
    app/services/notifications.py \
    app/services/audit.py \
    app/static/css/app.css \
    app/static/img/favicon.svg \
    app/templates/base.html \
    app/templates/layouts \
    app/templates/partials \
    app/templates/components \
    app/static/js/app.js \
    app/static/js/assistant.js \
    app/templates/errors \
    .github/ISSUE_TEMPLATE \
    .github/PULL_REQUEST_TEMPLATE.md \
  | tar xf - -C ~/collegehub

cd ~/collegehub
git status      # only this stage's files
```

```
git add app/__init__.py app/config.py app/extensions.py app/routes/__init__.py \
    app/services/__init__.py
git commit -m "feat: add app factory, config and extensions"

git add app/constants.py app/models.py
git commit -m "feat: add constants, database models and constraints"

git add app/decorators.py app/utils.py app/cli.py app/services/errors.py \
    app/services/notifications.py app/services/audit.py
git commit -m "feat: add decorators, utils, CLI and shared services"

git add app/static/css/app.css app/static/img/favicon.svg
git commit -m "style: add design-system CSS and favicon"

git add app/templates/base.html app/templates/layouts app/templates/partials \
    app/templates/components
git commit -m "feat: add base template, layouts, partials and macros"

git add app/static/js/app.js app/static/js/assistant.js app/templates/errors
git commit -m "feat: add shared JavaScript and styled error pages"

git add .github/ISSUE_TEMPLATE .github/PULL_REQUEST_TEMPLATE.md
git commit -m "chore: add issue templates and pull request template"
```

```
git push -u origin feature/backend-core
```

GitHub → **Compare & pull request** (base **main**) → read **Files changed** → **Squash and merge** (or *Create a merge commit*) → **Delete branch**. Then:

```
cd ~/collegehub
git checkout main
git pull
git branch -D feature/backend-core
```

(*-D* is needed after a squash merge; it removes only your local copy of the branch.)

CODEOWNERS: the file in this stage's commit contains a placeholder. Open *~/collegehub/.github/CODEOWNERS* after copying and replace *@your-github-username* with your real username before committing (stage 7 includes it; do it there).

Stage 2: Accounts, authentication, verification and public pages

Branch **feature/accounts-and-auth**

```
cd ~/collegehub
git checkout main
git pull
git checkout -b feature/accounts-and-auth

cd ~/collegehub-source
tar cf - \
    app/services/throttle.py \
    app/services/mailer.py \
    app/services/accounts.py \
    app/services/verification.py \
    app/routes/auth.py \
    app/templates/auth \
    app/templates/public \
  | tar xf - -C ~/collegehub

cd ~/collegehub
git status      # only this stage's files
```

```
git add app/services/throttle.py app/services/mailer.py
git commit -m "feat: add throttle and mailer services"

git add app/services/accounts.py
git commit -m "feat: add accounts service (registration, passwords, staff hierarchy)"

git add app/services/verification.py
git commit -m "feat: add member verification service"

git add app/routes/auth.py
git commit -m "feat: add authentication routes"

git add app/templates/auth
git commit -m "feat: add login, register and password pages"

git add app/templates/public
git commit -m "feat: add landing, events list, event detail and verify pages"
```

```
git push -u origin feature/accounts-and-auth
```

GitHub → **Compare & pull request** (base **main**) → read **Files changed** → **Squash and merge** (or *Create a merge commit*) → **Delete branch**. Then:

```
cd ~/collegehub
git checkout main
git pull
git branch -D feature/accounts-and-auth
```

(*-D* is needed after a squash merge; it removes only your local copy of the branch.)

Stage 3: Organizer and Faculty workspaces

Branch **feature/role-workspaces**

```
cd ~/collegehub
git checkout main
git pull
git checkout -b feature/role-workspaces

cd ~/collegehub-source
tar cf - \
    app/services/logistics.py \
    app/services/volunteering.py \
    app/routes/organizer.py \
    app/templates/organizer \
    app/services/announcements.py \
    app/services/analytics.py \
    app/routes/faculty.py \
    app/templates/faculty \
  | tar xf - -C ~/collegehub

cd ~/collegehub
git status      # only this stage's files
```

```
git add app/services/logistics.py app/services/volunteering.py
git commit -m "feat: add logistics and volunteering services"

git add app/routes/organizer.py
git commit -m "feat: add organizer routes"

git add app/templates/organizer
git commit -m \
  "feat: add organizer templates (events, analytics, member verification, logistics account)"

git add app/services/announcements.py app/services/analytics.py
git commit -m "feat: add announcements and analytics services"

git add app/routes/faculty.py
git commit -m "feat: add faculty routes (proposals, monitoring, organizer account)"

git add app/templates/faculty
git commit -m "feat: add faculty templates"
```

```
git push -u origin feature/role-workspaces
```

GitHub → **Compare & pull request** (base **main**) → read **Files changed** → **Squash and merge** (or *Create a merge commit*) → **Delete branch**. Then:

```
cd ~/collegehub
git checkout main
git pull
git branch -D feature/role-workspaces
```

(*-D* is needed after a squash merge; it removes only your local copy of the branch.)

Stage 4: Events, QR attendance, student area, logistics, AI

Branch **feature/events-qr-ai**

```
cd ~/collegehub
git checkout main
git pull
git checkout -b feature/events-qr-ai

cd ~/collegehub-source
tar cf - \
    app/services/events.py \
    app/services/registration.py \
    app/services/qr.py \
    app/services/attendance.py \
    app/services/achievements.py \
    app/services/certificates.py \
    app/routes/main.py \
    app/routes/api.py \
    app/routes/student.py \
    app/templates/student \
    app/templates/shared \
    app/routes/logistics.py \
    app/templates/logistics \
    app/static/js/scanner.js \
    app/services/documents.py \
    app/services/ai/__init__.py \
    app/services/ai/extraction.py \
    app/services/ai/processing.py \
    app/services/ai/retrieval.py \
    app/services/ai/provider.py \
    app/services/ai/validation.py \
    app/services/ai/assistant.py \
  | tar xf - -C ~/collegehub

cd ~/collegehub
git status      # only this stage's files
```

```
git add app/services/events.py app/services/registration.py
git commit -m "feat: add event lifecycle and registration services"

git add app/services/qr.py app/services/attendance.py
git commit -m "feat: add signed QR tokens and attendance service"

git add app/services/achievements.py app/services/certificates.py
git commit -m "feat: add achievements and certificate services"

git add app/routes/main.py app/routes/api.py
git commit -m "feat: add main and API routes"

git add app/routes/student.py
git commit -m "feat: add student routes"

git add app/templates/student app/templates/shared
git commit -m "feat: add student templates and notifications page"

git add app/routes/logistics.py
git commit -m "feat: add logistics routes"

git add app/templates/logistics app/static/js/scanner.js
git commit -m "feat: add logistics templates and QR scanner script"

git add app/services/documents.py app/services/ai/__init__.py \
    app/services/ai/extraction.py app/services/ai/processing.py
git commit -m "feat: add PDF document upload and text extraction"

git add app/services/ai/retrieval.py app/services/ai/provider.py \
    app/services/ai/validation.py app/services/ai/assistant.py
git commit -m "feat: add AI retrieval, provider, validation and assistant"
```

```
git push -u origin feature/events-qr-ai
```

GitHub → **Compare & pull request** (base **main**) → read **Files changed** → **Squash and merge** (or *Create a merge commit*) → **Delete branch**. Then:

```
cd ~/collegehub
git checkout main
git pull
git branch -D feature/events-qr-ai
```

(*-D* is needed after a squash merge; it removes only your local copy of the branch.)

Stage 5: Demo seed data

Branch **feature/seed-data**

```
cd ~/collegehub
git checkout main
git pull
git checkout -b feature/seed-data

cd ~/collegehub-source
tar cf - \
    app/seed.py \
  | tar xf - -C ~/collegehub

cd ~/collegehub
git status      # only this stage's files
```

```
git add app/seed.py
git commit -m "feat: add demo seed data with accounts in every verification state"
```

```
git push -u origin feature/seed-data
```

GitHub → **Compare & pull request** (base **main**) → read **Files changed** → **Squash and merge** (or *Create a merge commit*) → **Delete branch**. Then:

```
cd ~/collegehub
git checkout main
git pull
git branch -D feature/seed-data
```

(*-D* is needed after a squash merge; it removes only your local copy of the branch.)

Stage 6: Automated tests and CI

Branch **feature/tests-and-ci**

```
cd ~/collegehub
git checkout main
git pull
git checkout -b feature/tests-and-ci

cd ~/collegehub-source
tar cf - \
    tests/__init__.py \
    tests/conftest.py \
    tests/test_auth.py \
    tests/test_accounts.py \
    tests/test_verification.py \
    tests/test_profile.py \
    tests/test_credit.py \
    tests/test_events.py \
    tests/test_registration.py \
    tests/test_attendance.py \
    tests/test_authorization.py \
    tests/test_completion_volunteers.py \
    tests/test_ai_documents.py \
    tests/test_logistics_certificates.py \
    .github/workflows/ci.yml \
  | tar xf - -C ~/collegehub

cd ~/collegehub
git status      # only this stage's files
```

```
git add tests/__init__.py tests/conftest.py tests/test_auth.py \
    tests/test_accounts.py tests/test_verification.py tests/test_profile.py \
    tests/test_credit.py
git commit -m "test: add pytest fixtures and authentication/account/verification tests"

git add tests/test_events.py tests/test_registration.py tests/test_attendance.py \
    tests/test_authorization.py tests/test_completion_volunteers.py
git commit -m "test: add event, registration, attendance and authorization tests"

git add tests/test_ai_documents.py tests/test_logistics_certificates.py
git commit -m "test: add AI, logistics and certificate tests"

git add .github/workflows/ci.yml
git commit -m "chore: add GitHub Actions CI workflow"
```

```
git push -u origin feature/tests-and-ci
```

GitHub → **Compare & pull request** (base **main**) → read **Files changed** → **Squash and merge** (or *Create a merge commit*) → **Delete branch**. Then:

```
cd ~/collegehub
git checkout main
git pull
git branch -D feature/tests-and-ci
```

(*-D* is needed after a squash merge; it removes only your local copy of the branch.)

After this merge, the **Actions** tab runs the **CI** workflow (pyflakes and pytest) on every push and pull request. If it turns red, open the run, read the failing step, fix on a new *fix/…* branch.

Stage 7: Documentation and GitHub templates

Branch **docs/documentation**

```
cd ~/collegehub
git checkout main
git pull
git checkout -b docs/documentation

cd ~/collegehub-source
tar cf - \
    .github/CODEOWNERS \
    README.md \
    docs/DEVELOPER_MANUAL.md \
    docs/ARCHITECTURE.md \
    docs/DATABASE.md \
    docs/USER_MANUAL.md \
    docs/DEPLOYMENT_MANUAL.md \
    docs/GITHUB_MANUAL.md \
    docs/TESTING_CHECKLIST.md \
    docs/SUBMISSION_CHECKLIST.md \
    docs/FIVE_DAY_PLAN.md \
    docs/DEMO_SCRIPT.md \
  | tar xf - -C ~/collegehub

cd ~/collegehub
git status      # only this stage's files
```

```
git add .github/CODEOWNERS
git commit -m "chore: add CODEOWNERS"

git add README.md
git commit -m "docs: add README"

git add docs/DEVELOPER_MANUAL.md docs/ARCHITECTURE.md docs/DATABASE.md
git commit -m "docs: add developer manual, architecture and database guides"

git add docs/USER_MANUAL.md
git commit -m "docs: add user manual"

git add docs/DEPLOYMENT_MANUAL.md docs/GITHUB_MANUAL.md
git commit -m "docs: add deployment and GitHub manuals"

git add docs/TESTING_CHECKLIST.md docs/SUBMISSION_CHECKLIST.md
git commit -m "docs: add testing and submission checklists"

git add docs/FIVE_DAY_PLAN.md docs/DEMO_SCRIPT.md
git commit -m "docs: add five-day plan and demo script"
```

```
git push -u origin docs/documentation
```

GitHub → **Compare & pull request** (base **main**) → read **Files changed** → **Squash and merge** (or *Create a merge commit*) → **Delete branch**. Then:

```
cd ~/collegehub
git checkout main
git pull
git branch -D docs/documentation
```

(*-D* is needed after a squash merge; it removes only your local copy of the branch.)

1.7 Check that GitHub holds a working project

```
cd ~ && rm -rf collegehub-check
git clone https://github.com/<YOUR-USERNAME>/collegehub.git collegehub-check
cd collegehub-check
python -m venv .venv && source .venv/bin/activate     # Git Bash: source .venv/Scripts/activate
pip install -r requirements.txt pyflakes
cp .env.example .env
python -m pyflakes app tests
python -m pytest -q
```

Expect **244 passed** (it takes a couple of minutes). If the number differs, trust what you see, not this PDF. Then tag the release:

```
cd ~/collegehub
git checkout main && git pull
git tag -a v1.0 -m "CollegeHub final submission"
git push origin v1.0
```

Part 2. Run it on your laptop (recommended for the presentation)

```
cd ~/collegehub
python -m venv .venv
source .venv/bin/activate        # Git Bash: source .venv/Scripts/activate
pip install -r requirements.txt
cp .env.example .env
python -c "import secrets; print(secrets.token_hex(32))"      # copy the output
# open .env in an editor: paste it after SECRET_KEY= and set your own DEMO_PASSWORD

flask --app wsgi seed --reset
flask --app wsgi run --debug
```

Open **http://127.0.0.1:5000**. Demo logins (password = your *DEMO_PASSWORD*, default *CollegeHub@2026*):

Role	Username
Faculty Mentor	collegehub_faculty
Organizer	collegehub_org
Logistics	collegehub_logistics
Verified student	aisha.khan
Pending / rejected / suspended student	naveed.rashid / sumaira.hamid / irfan.peer

Show it on a phone (same Wi-Fi)

```
flask --app wsgi run --host 0.0.0.0 --port 5000
# find your laptop's IP: ipconfig (Windows) or ifconfig / ip a (macOS, Linux)
# phone browser:  http://<laptop-ip>:5000
```

The browser only allows the **QR camera** on *localhost* or HTTPS. On a phone over plain HTTP use the manual Registration-ID box in the scanner, or do the scanning from the laptop's own browser on *localhost*. Allow the laptop's firewall prompt for Python if the phone cannot connect.

Production-like local run

```
export APP_ENV=production SECRET_KEY=<your-long-random-string>     # PowerShell: $env:NAME=...
gunicorn wsgi:app --workers 2        # Linux/macOS
# Windows:  pip install waitress  then  waitress-serve --port 5000 wsgi:app
```

Production mode marks cookies *Secure*, so logging in over plain *http://* will not keep you signed in. Use *--debug* mode for local demos, production mode only behind HTTPS.

Part 3. Publish it free: Render + Neon

What the free tiers give you (checked 21 October 2026)

	Render free web service	Neon free database
Idle	Spins down after 15 minutes without traffic; waking takes about 1 minute	Compute suspends after 5 minutes idle (cannot be turned off); first query is slower
Quota	750 free instance hours per workspace per month; services are suspended when used up	1 GB storage per project, 100 compute-unit-hours per project, no credit card, permanent plan
Files	Ephemeral: every restart, redeploy and spin-down erases local files	Not applicable (database)
Other	No shell, no one-off jobs; SMTP ports 25, 465, 587 blocked; rollback only to the last two deploys; may restart at any time	Free Postgres from Render itself expires after 30 days, which is why Neon is used

What this means for CollegeHub, honestly

- **Database:** must be PostgreSQL on Neon. SQLite on Render would be wiped at each spin-down.
- **Uploads:** uploaded PDFs and cover images disappear on restart. The text extracted from PDFs is stored in the database, so the **AI assistant keeps answering**, but the PDF *Download* link and cover pictures break. Upload them shortly before presenting.
- **Password-reset e-mail cannot be sent** from Render free (SMTP is blocked); links are only written to the log. For a short supervised demo you may set *DEMO_SHOW_RESET_LINK=1* to show the link on screen, but that lets anyone who knows an e-mail reset that account: turn it off right after.
- **Cold start:** open the site once a minute before you present.
- Render's own documentation says free services are not meant for production use. For a real college deployment use a paid plan or the college's server.

Step 1: Neon database

Sign up at **neon.com** (no card), create a project (pick the region nearest your Render region), then copy the **connection string**. It looks like *postgresql://user:password@host/dbname?sslmode=require*. Treat it like a password: never put it in Git, a screenshot or a chat.

Step 2: put data in the database from your laptop

Render's free tier has no shell, so you run the setup commands on your laptop, pointed at Neon. **Choose one**:

(a) Public demo with sample data (fake accounts that share one password, so pick a password that is not used anywhere else):

```
cd ~/collegehub && source .venv/bin/activate
export DATABASE_URL="<your Neon connection string>"      # PowerShell: $env:DATABASE_URL="..."
export DEMO_PASSWORD="<a password you choose>"
export SECRET_KEY="<any long random string>"
flask --app wsgi seed
```

(b) Real use, no sample data: create only the Faculty Mentor. You will be asked for the password (typed twice, not shown). Then log in and create the Organizer from *Organizer account*; the Organizer creates the Logistics account.

```
export DATABASE_URL="<your Neon connection string>"
export SECRET_KEY="<any long random string>"
flask --app wsgi init-db
flask --app wsgi create-faculty --username <your_username> \
    --email <your e-mail> --name "Your Name"
```

Warning: *seed --reset* drops every table. Never use it on a database that holds data you want to keep.

Step 3: Render web service

- Push your code to GitHub first (Part 1); check that *.env* and *instance/* are not in the repository.
- render.com → sign in with GitHub → **New** → **Web Service** → choose the **collegehub** repository.
- Name: *collegehub* (your link becomes *https://collegehub.onrender.com* if the name is free). Region: near your Neon region. Branch: **main**. Runtime: **Python 3**. Instance type: **Free**.
- Build command: **pip install -r requirements.txt**
- Start command: **gunicorn wsgi:app --workers 2 --timeout 60**
- Add the environment variables in the table below, then press **Create Web Service** and watch the **Logs** until it says the service is live.

Key	Value
APP_ENV	production
SECRET_KEY	a long random string: python -c "import secrets; print(secrets.token_hex(32))" (the app refuses to start in production without one)
DATABASE_URL	the Neon connection string
PYTHON_VERSION	a 3.11+ version listed in Render's Python docs (for example 3.12.x); if the build fails on Python, check that page
APP_TIMEZONE	Asia/Kolkata
AI_API_KEY	optional. Leave it out for offline demo mode (the assistant still works, extractively)
DEMO_SHOW_RESET_LINK	leave unset (see the warning above)

Step 4: check it

Open *https://<your-service>.onrender.com/healthz*: it must show *{"status":"ok"}*. Then open the home page, log in as one of your accounts and click through: events, registration, and the footer (*Made with love by Aarif Mushtaq*). The first load after idle takes about a minute.

Updating the live site

```
cd ~/collegehub
git checkout -b fix/<topic>
# ...edit, test locally...
git add <files>
git commit -m "fix: <what you changed>"
git push -u origin fix/<topic>
# merge the pull request on GitHub: Render redeploys main automatically
```

Database changes: this project has no migrations (*db.create_all()* only creates missing tables, it never alters existing ones). If you change a model after the database holds data, re-seed a demo database or add Alembic. Rollback: Render → your service → **Events/Deploys** → redeploy an earlier deploy (only the last two are kept).

Part 4. Checklists and troubleshooting

Before the demo (day before)

- Run *python -m pytest -q* on the exact code you will present; write down the real number.
- *flask --app wsgi seed --reset* on the demo database (local) so every account is in a known state.
- Walk through *docs/DEMO_SCRIPT.md* twice with a timer.
- Phone charged; QR page open; camera permission tested on *localhost* (or manual-entry fallback ready).
- Hosted site: open it one minute before you start so it is awake. Keep the local copy ready as a backup.
- Screenshots of the main screens saved in case the network fails.
- Footer check: the credit **Made with love by Aarif Mushtaq** shows on the public pages and at the bottom of every signed-in page.

Security checklist (any public deployment)

- *APP_ENV=production* and a strong *SECRET_KEY* set only in Render's environment screen.
- Demo password is not a password you use anywhere else, or the demo accounts are removed after the demo.
- *DEMO_SHOW_RESET_LINK* is unset.
- No *.env*, database, upload or API key in the repository (*git ls-files | grep -E "\.env$|\.db$"* prints nothing).
- The Neon connection string and any token exist only in Render's environment screen and your own shell, never in a file in Git.
- If a secret was ever pushed: **rotate it immediately** (new database password in Neon, new SECRET_KEY, new token). Deleting the file later does not remove it from Git history.

Troubleshooting

Build fails while installing psycopg2

The project uses *psycopg2-binary*, which normally installs as a ready-made wheel. If the build uses a very new Python version without a wheel, set *PYTHON_VERSION* to a slightly older supported 3.11 or 3.12 release and redeploy.

“SECRET_KEY must be set to a strong random value in production”

Add *SECRET_KEY* in Render → Environment, then redeploy.

Login seems to work but I am sent back to the login page

In production the session cookie is *Secure* and needs HTTPS. Use the *https://* Render address, not *http://*. Running production mode locally over plain http causes the same.

500 error or “could not connect to server”

Check *DATABASE_URL* (copy it again from Neon; it must end with *sslmode=require*), and that the Neon project is not deleted. The first request after idle may be slow while the database wakes up; retry once.

“relation users does not exist” or missing column

The tables were never created, or the database was created by an older version of the project. Run *flask --app wsgi init-db* against Neon, or *seed --reset* on a demo database.

The site is very slow the first time

Render free had spun down. Wait about a minute; open it before you present.

PDF downloads and cover images are broken

Expected on Render free: files are erased on restart. Upload them again just before presenting. The AI assistant still answers because the text is stored in the database.

Password-reset link never arrives

Render free blocks outgoing SMTP. Read the link in Render's **Logs**, or reset the student's password with the Organizer tools; see the warning about *DEMO_SHOW_RESET_LINK* above.

Git: “Permission denied” or “Authentication failed”

Run *gh auth status* then *gh auth login*; or create a new personal access token. Make sure the repository URL has your own username.

Git: I committed .env or a password

If not pushed yet: *git reset --soft HEAD~1*, unstage the file (*git restore --staged .env*) and commit again. If already pushed: treat the secret as stolen and rotate it first.

Sources checked 21 October 2026: render.com/docs/free; neon.com/pricing. Limits change; re-check them before you rely on them.